

UNIVERSITÉ MOHAMMED PREMIER
École Nationale des Sciences Appliquées d'Oujda
Filière Génie Informatique – GI3

RAPPORT DE MINI-PROJET

Gestion de Bibliothèque (Livres et Emprunts)

Application : bibliothequeApp

Réalisé par :

Fayza Belhaj : Backend & Base de données
Kaoutar Baladi : Frontend & Interface graphique

Encadrante : Mme Douae EL HILA
Professeure du module : Mme Ilham ELFARISSI

Module : Développement Java IHM
Année universitaire : 2025/2026

Remerciements

Ce projet a été une expérience enrichissante qui nous a permis de mettre en pratique nos compétences en développement Java et en conception d'interfaces utilisateur. Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Nous adressons nos plus vifs remerciements à **Mme Douae EL HILA**, notre encadrante de projet, pour sa disponibilité, ses précieux conseils et son accompagnement tout au long de ce projet. Sa rigueur et son expertise nous ont guidés dans les moments difficiles et nous ont permis d'aboutir à une application de qualité.

Nous tenons également à remercier chaleureusement **Mme Ilham ELFARISSI**, professeure du module "Développement Java IHM", pour la qualité de son enseignement et les connaissances fondamentales qu'elle nous a transmises. Ses cours nous ont fourni les bases solides nécessaires à la réalisation de ce projet.

Nos remerciements s'adressent également à toute l'équipe pédagogique du département Génie Informatique de l'ENSAO pour leur soutien et leur engagement envers la formation des étudiants.

Enfin, nous tenons à nous remercier mutuellement pour notre excellente collaboration tout au long de ce projet. Le travail d'équipe, la communication et l'entraide ont été les clés de notre réussite. Chacun a apporté ses compétences et son savoir-faire pour aboutir à une application dont nous sommes fiers.

À toutes et à tous, merci.

Table des matières

Remerciements	2
Introduction	5
1. Présentation du projet	6
1.1 Description del'application	6
1.2 Besoins fonctionnels	6
1.3 Besoins non fonctionnels	7
2. Conception	
2.1 Architecture MVC	8
2.2 Diagramme de classes UML	8
2.3 Diagramme de cas d'utilisation	10
2.4 Diagramme de séquence(ajoutd'unlivre)	10
3. Environnement de travail	12
3.1 Outilset technologies utilisés	12
3.2 Structure du projet	12
3.3 Configuration dela base dedonnées	12
4. Répartition des tâches	14
4.1 Tâches de l'étudiant 1 – Kaoutar Baladi	14
4.2 Tâches de l'étudiant 2 – Fayza Belhaj	14
5. Explication du code	16
5.1 Connexion à labase de données–Database.java	16
5.2 Les classes modèles	16
5.3 Les classes DAO	16
5.4 Tests et validation des classes DAO	17
5.5 Les contrôleurs – MainController.java	18
5.6 Les fichiers FXML – main-view.fxml	18
5.7 Fonctionnalité spécifiqueauprojet–ExportCSV	19
6. Interfaces de l'application	20
6.1 Fenêtrprincipale	20
6.2 Interfaces de gestion de l'entité 1 — Livres	20
6.3 Interfaces de gestion de l'entité 2 — Emprunts	21
6.4 Tableau de bord — Statistiques	21
6.5 Menus et dialogues	22
6.6 Export de données	22
7. Tests utilisateur et démarche qualité	24
7.1 Scénarios detestfonctionnels	24

7.2 Démarche de gestion de version.....	24
7.3 Bilan de la démarche qualité.....	25
Conclusion.....	26
Glossaire.....	28
Références.....	29

Introduction

Dans le cadre du module Développement Java IHM dispensé en troisième année de la filière Génie Informatique à l'École Nationale des Sciences Appliquées d'Oujda, il nous a été demandé de concevoir et de développer une application graphique complète mettant en pratique les concepts fondamentaux de la programmation orientée objet, de la conception d'interfaces utilisateur et de la gestion de bases de données relationnelles.

Le projet que nous avons choisi de réaliser s'intitule « Gestion de Bibliothèque (Livres et Emprunts) », développé sous le nom d'application bibliothequeApp. Il s'agit d'une application de bureau développée avec la technologie JavaFX, destinée à faciliter la gestion quotidienne d'une bibliothèque, en particulier le suivi du catalogue des ouvrages disponibles et le suivi des emprunts effectués par les usagers.

La gestion manuelle des bibliothèques, à l'aide de registres papier ou de simples feuilles de calcul, présente de nombreuses limites : risque de perte d'informations, difficulté à retrouver rapidement un livre, absence de statistiques fiables sur l'état du fonds documentaire, et gestion fastidieuse des retards de retour. Notre application vise à répondre à ces problématiques en proposant une solution informatique simple, robuste et ergonomique.

Ce rapport présente l'ensemble du travail réalisé dans le cadre de ce mini-projet. Il décrit dans un premier temps le contexte et les objectifs du projet, puis détaille la phase de conception (architecture MVC, diagrammes UML), l'environnement technique utilisé, la répartition des tâches au sein du binôme, l'implémentation du code source et enfin les interfaces graphiques obtenues. Une conclusion générale clôturera ce document en dressant un bilan du travail accompli et en proposant des pistes d'amélioration futures.

Présentation du projet

1.1 Description de l'application

L'application « Gestion de Bibliothèque », nommée *bibliothequeApp*, est un logiciel de bureau développé en Java avec l'interface graphique JavaFX, couplé à une base de données MySQL pour la persistance des informations. Elle s'adresse principalement à un bibliothécaire ou à un gestionnaire de fonds documentaire, qui dispose ainsi d'un outil centralisé pour suivre l'ensemble du cycle de vie des livres et des emprunts.

L'application repose sur deux entités principales autour desquelles s'organise l'ensemble des fonctionnalités :

- **Livre** : représente un ouvrage du fonds documentaire de la bibliothèque, caractérisé par son titre, son auteur, son ISBN, sa catégorie, son année de publication, son nombre de pages, son prix, sa note d'appréciation, son état physique, sa disponibilité, une description ainsi que sa date de publication.
- **Emprunt** : représente une transaction de prêt entre la bibliothèque et un usager. Il est caractérisé par le livre emprunté, le nom de l'emprunteur, la date d'emprunt, la date de retour prévue, la date de retour réelle, le statut de l'emprunt (en cours ou terminé) ainsi que des remarques éventuelles.

Grâce à cette modélisation, l'application permet de garder une trace précise de chaque ouvrage et de chaque mouvement de prêt, tout en facilitant la recherche, le filtrage et l'analyse statistique des données.

En définitive, l'application se positionne comme un outil métier complet, couvrant l'ensemble du cycle de vie documentaire : de l'acquisition d'un nouvel ouvrage jusqu'à son retrait éventuel du catalogue, en passant par les multiples cycles d'emprunt et de retour qu'il peut connaître durant sa présence dans le fonds. Elle a été pensée comme une brique logicielle autonome, ne nécessitant aucune connexion Internet pour fonctionner, ce qui la rend particulièrement adaptée à un usage dans des structures disposant d'une infrastructure réseau limitée, telles que les bibliothèques scolaires ou associatives de petite taille.

Le choix de la technologie JavaFX pour la réalisation de l'interface graphique a été motivé par plusieurs considérations pédagogiques et techniques. JavaFX, en tant que successeur de Swing dans l'écosystème Java, offre un modèle de programmation moderne basé sur des fichiers FXML déclaratifs, un système de liaison de données (data binding) puissant, ainsi qu'une personnalisation visuelle aisée à travers des feuilles de style CSS, ce qui correspond précisément aux objectifs pédagogiques du module Développement Java IHM.

1.2 Besoins fonctionnels

L'analyse des besoins exprimés pour ce projet a permis d'identifier les fonctionnalités suivantes, indispensables au bon fonctionnement de l'application :

- **Gestion complète (CRUD) des livres** : ajout, consultation, modification et suppression des ouvrages.

- Gestion complète (CRUD) des emprunts : création, consultation, modification et suppression des emprunts.
 - Recherche de livres par titre, par auteur ou par numéro ISBN.
 - Filtrage des livres par catégorie et par disponibilité.
 - Calcul et affichage de statistiques globales : nombre total de livres, livres disponibles, livres empruntés, note moyenne, taux de disponibilité et répartition par catégorie.
 - Export des données du catalogue au format CSV pour une utilisation externe (Excel, tableurs, etc.).
- Organisation de l'interface en plusieurs onglets afin de séparer clairement la gestion des livres, la gestion des emprunts et le tableau de bord statistique.

Cette analyse des besoins a été menée conjointement par les deux membres du binôme dès le début du projet, en s'appuyant à la fois sur les exigences explicites du cahier des charges du module et sur une réflexion sur les usages réels d'une petite bibliothèque (établissement scolaire, club de lecture, médiathèque associative). Elle a permis de prioriser les fonctionnalités essentielles — gestion des livres et des emprunts — avant d'envisager les fonctionnalités complémentaires telles que l'export CSV ou le tableau de bord statistique.

1.3 Besoins non fonctionnels

Outre les fonctionnalités métier, plusieurs exigences non fonctionnelles ont guidé la conception et le développement de l'application :

- Ergonomie : interface intuitive, claire et agréable, permettant une prise en main rapide par un utilisateur non technique.
- Performance : temps de réponse rapide pour les opérations de recherche, de filtrage et d'affichage des données, même avec un volume de données conséquent.
- Fiabilité : validation systématique des données saisies afin d'éviter les incohérences dans la base de données.
- Portabilité : l'application doit pouvoir fonctionner sur tout système disposant d'une machine virtuelle Java (JVM) compatible.
- Maintenabilité : respect d'une architecture logicielle claire (MVC) et d'un découpage en couches (modèle, vue, contrôleur, accès aux données) facilitant l'évolution future du code.
- Sécurité minimale : gestion des exceptions et des erreurs de connexion à la base de données afin d'éviter les plantages de l'application.

Conception

2.1 Architecture MVC

Afin de structurer notre application de manière claire et maintenable, nous avons adopté le patron de conception MVC (Modèle - Vue - Contrôleur), largement utilisé dans le développement d'applications JavaFX. Ce patron permet de séparer distinctement les responsabilités de chaque composant logiciel :

- **Modèle (Model)** : représenté par les classes `Livre.java` et `Emprunt.java`, qui encapsulent les données métier de l'application ainsi que leur comportement propre (getters, setters, méthodes utilitaires).
- **Vue (View)** : représentée par le fichier `main-view.fxml`, qui décrit la structure de l'interface graphique, ainsi que la feuille de style `style.css`, qui définit l'apparence visuelle des composants.
- **Contrôleur (Controller)** : représenté par la classe `MainController.java`, qui fait le lien entre la vue et le modèle, en gérant les événements utilisateur (clics, sélections, saisies) et en orchestrant les appels aux classes DAO.

Cette séparation des préoccupations facilite grandement la collaboration au sein du binôme : le Membre A peut faire évoluer la logique métier et l'accès aux données sans impacter directement l'interface, tandis que le Membre B peut modifier l'apparence et l'ergonomie sans toucher à la logique de persistance.

À cette architecture s'ajoute le patron DAO (Data Access Object), qui isole l'ensemble des opérations d'accès à la base de données MySQL dans des classes dédiées (`LivreDAO`, `EmpruntDAO`, `StatistiqueDAO`), évitant ainsi de mélanger logique métier et requêtes SQL.

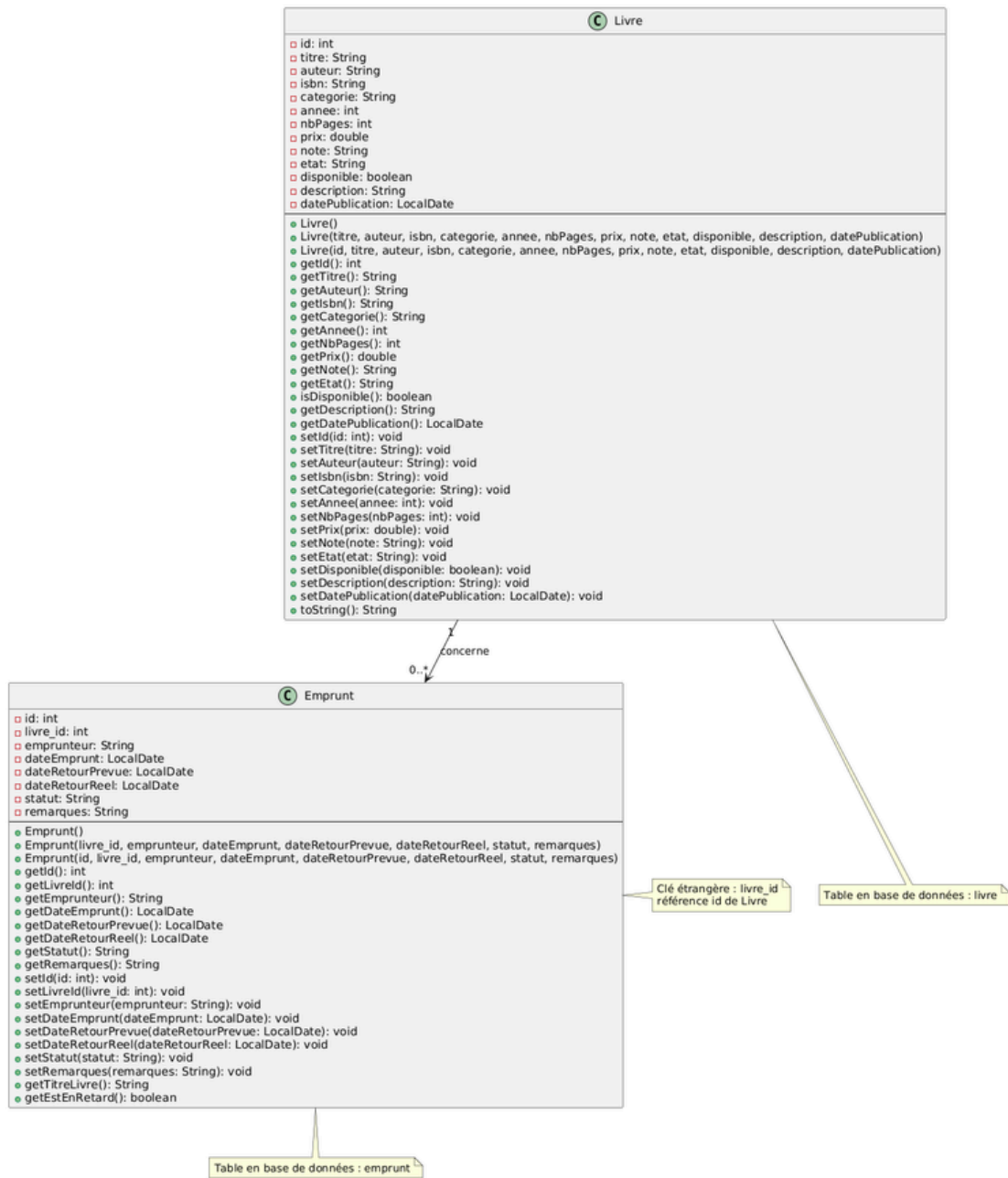
Ce découpage en couches successives — modèle, vue, contrôleur, accès aux données — présente plusieurs avantages majeurs pour un projet réalisé en binôme. D'une part, il permet de réduire le couplage entre les composants : une modification de la structure visuelle de l'interface n'impose pas de modifier la logique d'accès aux données, et inversement, une évolution du schéma de la base de données peut être absorbée par les classes DAO sans répercussion directe sur le contrôleur, à condition que les signatures des méthodes restent stables. D'autre part, cette architecture facilite très nettement les tests, puisque chaque couche peut être testée de manière isolée : les classes `TestDAO` et `TestEmpruntDAO` illustrent ce principe en validant le comportement des méthodes de persistance indépendamment de toute interface graphique.

Enfin, cette organisation en couches constitue une base solide pour l'évolutivité future de l'application. Il serait par exemple envisageable, sans remettre en cause l'architecture globale, de remplacer la couche de persistance MySQL par une autre technologie de stockage (PostgreSQL, SQLite, voire une API REST distante), ou de développer une nouvelle vue (par exemple une interface web) réutilisant directement les classes du modèle et les classes DAO existantes.

2.2 Diagramme de classes UML

Le diagramme de classes ci-dessous présente la structure statique du modèle de données de l'application, à savoir les classes `Livre` et `Emprunt` ainsi que la relation qui les unit.

Diagramme de Classes - Gestion de Bibliothèque



Classe	Attributs principaux
Livre	id, titre, auteur, isbn, categorie, annee, nbPages, prix, note, etat, disponible, description, datePublication
Emprunt	id, livre (référence), emprunteur, dateEmprunt, dateRetourPrevue, dateRetourReel, statut, remarques

Figure 2.1 – Attributs des classes du modèle

Chacune de ces deux classes expose, en plus de ses attributs privés, un ensemble de constructeurs (par défaut, avec paramètres partiels pour l'interface, complets pour la base de données) ainsi que des accesseurs et mutateurs (getters/setters) publics, conformément aux principes de l'encapsulation.

La relation entre les deux classes est une relation de composition/association : un livre peut faire l'objet de plusieurs emprunts au cours du temps (relation 1 vers plusieurs), tandis qu'un emprunt concerne toujours un seul et unique livre à un instant donné. Cette relation se traduit, au niveau de la base de données, par une clé étrangère livre_id dans la table emprunt référant la clé primaire id de la table livre.

Le tableau ci-dessous détaille, pour chaque attribut de la classe Livre, le type Java utilisé et une brève description de sa signification métier :

Attribut	Type	Description
id	String	Identifiant unique généré automatiquement par la base
titre	String	Titre complet de l'ouvrage
auteur	String	Nom de l'auteur ou des auteurs
isbn	String	Numéro international normalisé du livre
categorie	int	Genre ou catégorie thématique
annee	int	Année de première publication
nbPages	double	Nombre total de pages de l'ouvrage
prix	double	Prix d'acquisition ou de référence
note	String	Note d'appréciation, sur une échelle de 0 à 5
etat	boolean	État physique du livre (Neuf, Bon état, Usagé)
disponible	String	Indique si le livre peut être emprunté
description	LocalDate	Résumé ou commentaire libre
datePublication		Date précise de publication

Tableau 2.1 – Détail des attributs de la classe Livre

De manière similaire, la classe Emprunt regroupe les attributs nécessaires au suivi complet d'une transaction de prêt, depuis sa création jusqu'à sa clôture éventuelle, en passant par le suivi des retards. Le champ statut, encodé sous forme de chaîne de caractères (« En cours » ou « Terminé »), permet de filtrer aisément les emprunts actifs lors du calcul des statistiques.

2.3 Diagramme de cas d'utilisation

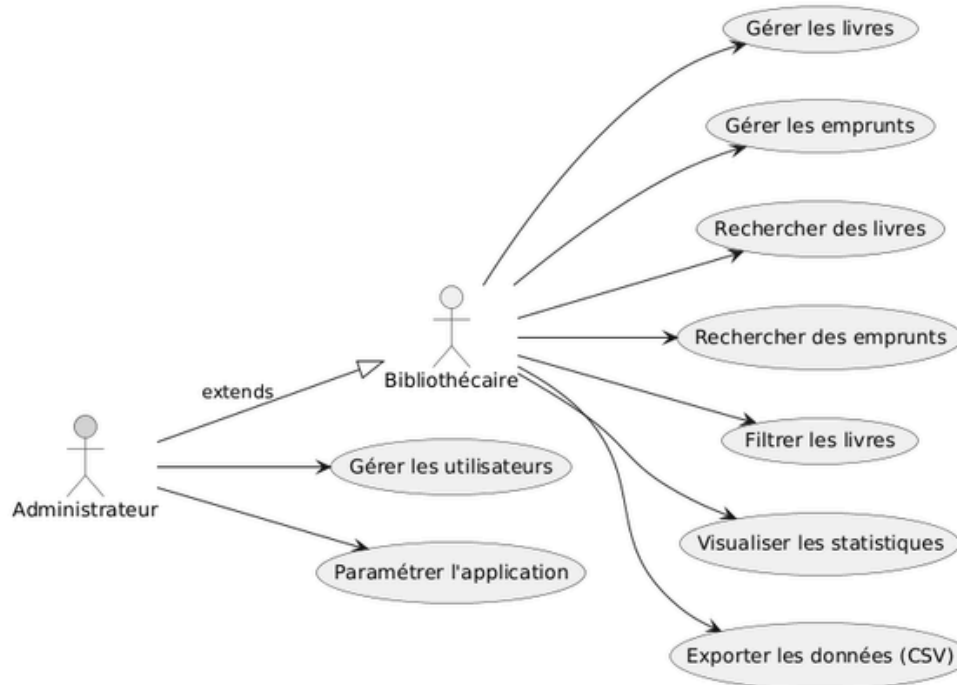
Le diagramme de cas d'utilisation permet d'identifier les principales interactions entre l'acteur du système — le bibliothécaire — et les fonctionnalités offertes par l'application.

- Acteur principal : Bibliothécaire.
- Gérer les livres : ajouter, modifier, supprimer et consulter les ouvrages du catalogue.
- Gérer les emprunts : créer, modifier, supprimer et consulter les emprunts en cours ou terminés.
- Rechercher : retrouver rapidement un livre par titre, auteur ou ISBN, ou un emprunt par nom d'emprunteur.
- Filtrer : affiner l'affichage du catalogue par catégorie ou par disponibilité.
- Visualiser les statistiques : consulter le tableau de bord présentant les indicateurs clés de la bibliothèque.
- Exporter les données : générer un fichier CSV contenant les informations du catalogue.

Tous ces cas d'utilisation sont accessibles depuis une interface unique organisée en onglets, ce qui simplifie la navigation pour l'utilisateur final.

Il convient de noter que certains cas d'utilisation entretiennent des relations d'inclusion entre eux. Par exemple, le cas d'utilisation « Gérer les emprunts » inclut implicitement une vérification de la disponibilité du livre concerné, opération qui repose elle-même sur les données maintenues par le cas d'utilisation « Gérer les livres ». De même, le cas d'utilisation « Visualiser les statistiques » dépend directement de l'état courant des données gérées par les deux cas d'utilisation précédents, ce qui illustre la forte cohésion fonctionnelle de l'application autour de son modèle de données central.

Bien qu'un seul acteur humain (le bibliothécaire) interagisse directement avec l'application dans la version actuelle, l'architecture mise en place permettrait, dans une évolution future, d'introduire un second acteur tel qu'un « Administrateur », disposant de droits supplémentaires (gestion des utilisateurs, paramétrage de l'application), sans remettre en cause la structure générale du diagramme de cas d'utilisation.

Diagramme de Cas d'Utilisation - Gestion de Bibliothèque**2.4 Diagramme de séquence (ajout d'un livre)**

Afin d'illustrer le fonctionnement interne de l'application, nous détaillons ci-dessous le scénario d'ajout d'un nouveau livre, qui met en jeu l'ensemble des couches de l'architecture MVC/DAO.

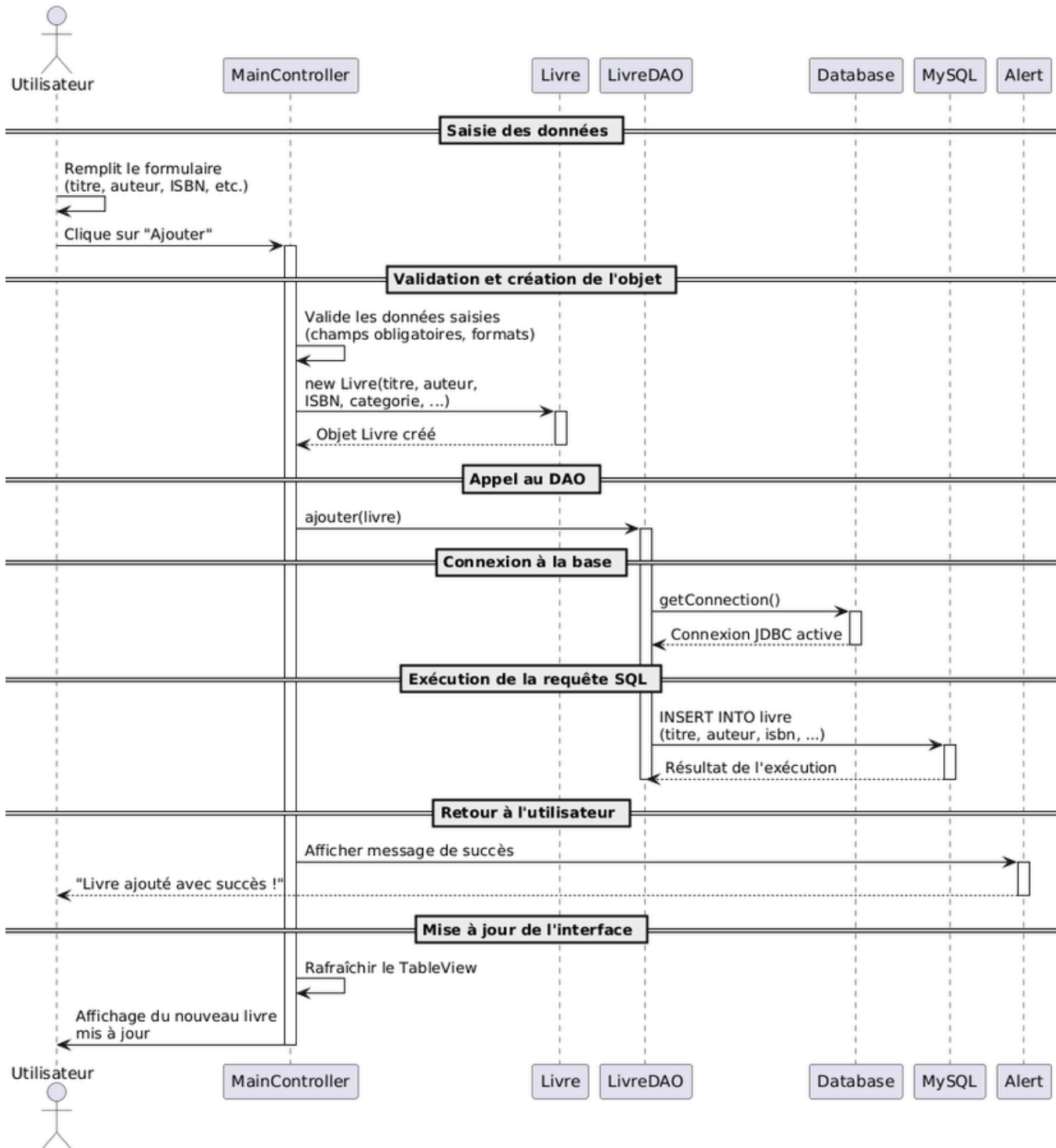
Utilisateur -> MainController -> LivreDAO -> Database -> MySQL

Le déroulement de ce scénario se décompose en huit étapes successives :

1. L'utilisateur remplit le formulaire de saisie d'un livre (titre, auteur, ISBN, catégorie, etc.) dans l'onglet « Livres ».
2. L'utilisateur clique sur le bouton « Ajouter ».
3. Le contrôleur MainController valide les données saisies (champs obligatoires, formats attendus).
4. Le contrôleur crée un nouvel objet Livre à partir des données validées.
5. Le contrôleur appelle la méthode ajouter() de la classe LivreDAO en lui passant l'objet Livre créé.
6. La classe Database établit (ou réutilise) une connexion JDBC active vers le serveur MySQL.
7. La requête SQL INSERT correspondante est exécutée sur la base de données MySQL.
8. Le résultat de l'opération est retourné à l'utilisateur via une boîte de dialogue Alert confirmant le succès, ou signalant une erreur en cas d'échec.

Ce mécanisme, identique dans son principe pour les opérations de modification et de suppression, garantit une cohérence entre l'interface graphique et les données réellement persistées dans la base MySQL.

Diagramme de Séquence - Ajout d'un livre



Environnement de travail

3.1 Outils et technologies utilisés

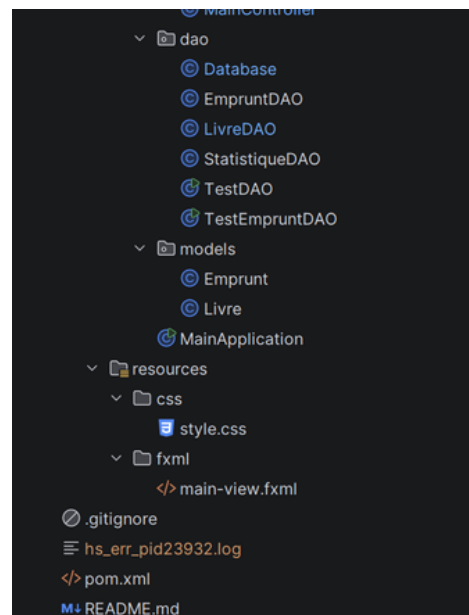
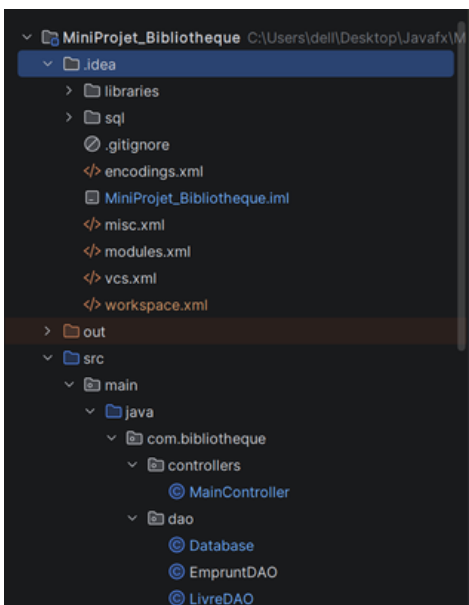
L'élaboration de ce mini-projet a nécessité la mise en place d'un environnement de développement complet, regroupant les outils suivants :

Outil / Technologie	Rôle dans le projet
Java (JDK 17)	Langage de programmation principal de l'application
JavaFX 17.0.6	Framework de construction de l'interface graphique
MySQL 8.0	Système de gestion de base de données relationnelle
WAMP	Serveur local hébergeant MySQL et l'environnement de test
IntelliJ IDEA 2026.1	Environnement de développement intégré (IDE)
Maven	Gestionnaire de dépendances et de build du projet
Git / GitHub	Versionnement du code source et collaboration en binôme
Scene Builder	Conception visuelle des fichiers FXML
Figma (optionnel)	Maquettage préliminaire de l'interface utilisateur

Tableau 3.1 – Outils et technologies utilisés

3.2 Structure du projet

Le projet a été organisé selon une arborescence Maven standard, respectant la séparation des couches modèle / vue / contrôleur / accès aux données, comme illustré ci-dessous :



Cette organisation modulaire respecte les conventions Maven et facilite la navigation dans le code, la maintenance ainsi que l'intégration de nouvelles fonctionnalités. Le fichier pom.xml centralise la déclaration des dépendances JavaFX et du connecteur JDBC MySQL, ainsi que la configuration du plugin de compilation.

3.3 Configuration de la base de données

La base de données `bibliotheque_db` a été créée localement à l'aide du serveur WAMP, qui regroupe en un seul environnement Apache, MySQL et phpMyAdmin. Le script `init_db.sql`, exécuté une seule fois lors de l'initialisation du projet, met en place les deux tables principales de l'application ainsi que la contrainte de clé étrangère reliant la table `emprunt` à la table `livre`.

```
CREATE DATABASE IF NOT EXISTS bibliotheque_db;
USE bibliotheque_db;

CREATE TABLE livre (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titre VARCHAR(150) NOT NULL,
  auteur VARCHAR(100) NOT NULL,
  isbn VARCHAR(20),
  categorie VARCHAR(50),
  annee INT,
  nb_pages INT,
  prix DOUBLE,
  note DOUBLE,
  etat VARCHAR(20),
  disponible BOOLEAN DEFAULT TRUE,
  description TEXT,
  date_publication DATE
);

CREATE TABLE emprunt (
  id INT AUTO_INCREMENT PRIMARY KEY,
  livre_id INT NOT NULL,
  emprunteur VARCHAR(100) NOT NULL,
  date_emprunt DATE,
  date_retour_prevue DATE,
  date_retour_reel DATE,
  statut VARCHAR(20),
  remarques TEXT,
  FOREIGN KEY (livre_id) REFERENCES livre(id) ON DELETE RESTRICT
);
```

Extrait 3.1 – Script de création des tables `init_db.sql`

Cette structure relationnelle garantit l'intégrité référentielle entre les deux tables : il est impossible d'enregistrer un emprunt référençant un livre inexistant, et la suppression d'un livre encore concerné par des emprunts actifs déclenche, côté application, un message d'avertissement avant toute opération.

Répartition des tâches

Conformément aux exigences du module, le travail a été réparti entre les deux membres du binôme selon leurs domaines de compétences respectifs : le Membre A s'est concentré sur le backend et la base de données, tandis que le Membre B s'est chargé du frontend et de l'interface graphique. Cette répartition a permis de paralléliser efficacement le développement tout en assurant une intégration cohérente du projet final.

4.1 Tâches de l'étudiant 1 – Fayza Belhaj

Tâche réalisée	Fichier(s) concerné(s)	Statut
Création et configuration du dépôt GitHub	Dépôt distant	Terminé
Rédaction du script SQL de création des tables	init_db.sql	Terminé
Connexion JDBC à la base MySQL	Database.java	Terminé
Modélisation de la classe Livre	Livre.java	Terminé
Modélisation de la classe Emprunt	Emprunt.java	Terminé
Implémentation du CRUD, recherche et filtrage des livres	LivreDAO.java	Terminé
Implémentation du CRUD et recherche des emprunts	EmpruntDAO.java	Terminé
Calcul des statistiques de la bibliothèque	StatistiqueDAO.java	Terminé
Rédaction de tests unitaires	TestDAO.java, TestEmpruntDAO.java	Terminé
Intégration des DAO avec les contrôleurs	MainController.java	Terminé
Gestion des exceptions et des cas limites	Ensemble du backend	Terminé

Tableau 4.1 – Tâches réalisées par l'étudiant 1

4.2 Tâches de l'étudiant 2 – Kaoutar Baladi

Tâche réalisée	Fichier(s) concerné(s)	Statut
Conception de la structure de l'interface graphique	main-view.fxml	Terminé
Réalisation de la feuille de style personnalisée	style.css	Terminé
Développement de la logique des	MainController.java	Terminé

Tâche réalisée	Fichier(s) concerné(s)	Statut
événements		
Intégration des 16+ contrôles JavaFX exigés	main-view.fxml	Terminé
Gestion des actions ajouter / modifier / supprimer / rechercher	MainController.java	Terminé
Implémentation de l'export CSV via FileChooser	MainController.java	Terminé
Conception de l'affichage visuel des statistiques	main-view.fxml	Terminé
Ajout des info-bulles (Tooltip) sur les champs	main-view.fxml	Terminé
Mise en place des alertes de confirmation et d'erreur	MainController.java	Terminé
Tests manuels de l'interface graphique	Application complète	Terminé
Réalisation des captures d'écran pour le rapport	Documentation	Terminé

Tableau 4.2 – Tâches réalisées par l'étudiant 2

Au-delà de cette répartition initiale, des points de synchronisation réguliers ont été organisés entre les deux membres afin de garantir la cohérence entre le modèle de données conçu côté backend et son exploitation côté interface graphique, notamment concernant le nommage des attributs et le format des données échangées.

Explication du code

5.1 Connexion à la base de données – Database.java

La classe Database constitue le point d'entrée unique vers la base de données MySQL. Elle implémente le patron de conception Singleton, garantissant qu'une seule instance de connexion est partagée par l'ensemble de l'application, ce qui évite la multiplication inutile de connexions JDBC et améliore les performances.

Les paramètres de connexion — URL du serveur, nom d'utilisateur et mot de passe — sont définis sous forme de constantes statiques au sein de la classe, facilitant ainsi leur modification en cas de changement d'environnement (passage de WAMP à un serveur de production, par exemple). La méthode getConnection() retourne une connexion JDBC active vers la base bibliotheque_db, tandis que la méthode closeConnection() permet de libérer proprement les ressources associées à la connexion lorsque celle-ci n'est plus nécessaire.

La gestion des exceptions SQLException est assurée à ce niveau afin d'informer l'utilisateur en cas d'échec de connexion (serveur MySQL non démarré, identifiants incorrects, base de données inexistante), via l'affichage d'une alerte explicite plutôt qu'un plantage brutal de l'application.

5.2 Les classes modèles

5.2.1 Livre.java

La classe Livre représente fidèlement un ouvrage du catalogue de la bibliothèque. Tous ses attributs (titre, auteur, isbn, categorie, annee, nbPages, prix, note, etat, disponible, description, datePublication) sont déclarés privés, conformément au principe d'encapsulation, et ne sont accessibles que via des méthodes getters et setters publiques. Plusieurs constructeurs sont proposés afin de couvrir les différents contextes d'utilisation : un constructeur vide pour l'instanciation depuis l'interface FXML, un constructeur partiel utilisé lors de l'ajout d'un nouveau livre (avant l'attribution de son identifiant par la base de données), et un constructeur complet utilisé lors de la récupération des données depuis MySQL.

5.2.2 Emprunt.java

La classe Emprunt modélise une opération de prêt. Elle contient une référence directe vers l'objet Livre concerné, ce qui permet d'accéder facilement aux informations du livre emprunté sans requête supplémentaire. En complément des attributs classiques (emprunteur, dateEmprunt, dateRetourPrevue, dateRetourReel, statut, remarques), la classe propose des méthodes utilitaires telles que getTitreLivre(), qui simplifie l'affichage du titre dans le TableView des emprunts, et getEstEnRetard(), qui compare la date courante à la date de retour prévue afin de signaler visuellement les emprunts en retard.

5.3 Les classes DAO

5.3.1 LivreDAO.java

La classe LivreDAO centralise l'ensemble des opérations de persistance relatives aux livres. Elle expose notamment les méthodes suivantes :

- ajouter(Livre livre) : insère un nouveau livre dans la table livre via une requête SQL INSERT préparée.
- listerTous() : retourne la liste complète des livres enregistrés dans la base, sous forme d'une liste d'objets Livre.
- modifier(Livre livre) : met à jour les informations d'un livre existant à partir de son identifiant.
- supprimer(int id) : supprime un livre de la base à partir de son identifiant unique.
- rechercher(String motCle) : retourne les livres dont le titre ou l'auteur contient le mot-clé fourni, à l'aide d'une clause SQL LIKE.
- filtrerParCategorie(String categorie) : retourne uniquement les livres appartenant à la catégorie sélectionnée.

5.3.2 EmpruntDAO.java

De manière analogue, la classe EmpruntDAO gère la persistance des emprunts à travers les méthodes suivantes :

- ajouter(Emprunt emprunt) : enregistre un nouvel emprunt et met à jour automatiquement la disponibilité du livre concerné.
 - listerTous() : retourne l'ensemble des emprunts enregistrés, accompagnés des informations du livre associé.
 - modifier(Emprunt emprunt) : met à jour un emprunt existant, notamment lors de l'enregistrement d'un retour.
 - supprimer(int id) : supprime un emprunt de la base de données.
 - rechercherParNom(String nom) : retourne les emprunts correspondant à un emprunteur donné.
- getEmpruntsEnCours() : retourne la liste des emprunts dont le statut est « En cours », utilisée notamment pour le calcul des statistiques.

Toutes les requêtes SQL exécutées par ces classes DAO utilisent des PreparedStatement, ce qui permet de se protéger efficacement contre les injections SQL tout en améliorant les performances grâce à la réutilisation du plan d'exécution des requêtes.

La classe StatistiqueDAO, quant à elle, regroupe les requêtes agrégées nécessaires au tableau de bord : comptage du nombre total de livres, comptage des livres disponibles via une clause WHERE disponible = true, calcul de la note moyenne à l'aide de la fonction SQL AVG(), et répartition du catalogue par catégorie grâce à une requête GROUP BY categorie. Ces calculs sont effectués directement côté base de données plutôt que côté application, ce qui réduit la quantité de données transférées et améliore les performances, en particulier lorsque le catalogue contient un grand nombre d'ouvrages.

5.4 Tests et validation des classes DAO

Afin de garantir la fiabilité des opérations de persistance, les classes TestDAO et TestEmpruntDAO ont été développées en parallèle de l'implémentation des classes LivreDAO et EmpruntDAO. Ces classes de test, exécutées indépendamment de l'interface graphique,

permettent de vérifier que chaque méthode CRUD se comporte conformément aux attentes : insertion correcte d'un nouvel enregistrement, récupération fidèle des données stockées, mise à jour effective lors d'une modification, et suppression définitive lors d'un retrait du catalogue.

Au-delà de ces tests automatisés, une phase de tests manuels a été menée conjointement par les deux membres du binôme sur l'ensemble de l'interface graphique, couvrant notamment la saisie de données invalides (champs vides, valeurs hors limites), la vérification du bon affichage des messages d'erreur, ainsi que le contrôle du comportement de l'application en cas de coupure de la connexion à la base de données pendant son utilisation.

5.5 Les contrôleurs–MainController.java

La classe MainController constitue le cœur logique de l'interface graphique. Implémentant l'interface Initializable de JavaFX, elle prend en charge l'ensemble des responsabilités suivantes :

- L'initialisation des composants graphiques lors du chargement de la vue, via la méthode `initialize()`.
- Le remplissage et la mise à jour dynamique des composants TableView affichant respectivement les livres et les emprunts.
- La gestion des événements déclenchés par les boutons d'ajout, de modification et de suppression, avec validation préalable des champs de saisie.
- La gestion de la recherche textuelle et du filtrage par catégorie ou disponibilité.
- Le rafraîchissement des indicateurs statistiques affichés dans l'onglet « Statistiques ».
- Le déclenchement de l'export des données au format CSV.

Pour limiter la taille et la complexité de cette classe, certaines responsabilités (calculs statistiques, accès aux données) ont été déléguées aux classes DAO dédiées, le contrôleur se contentant d'orchestrer les appels et de mettre à jour l'affichage en conséquence.

5.6 Les fichiers FXML–main-view.fxml

Le fichier main-view.fxml décrit, en langage déclaratif XML, l'ensemble de la structure visuelle de l'application. Il s'organise principalement autour des éléments suivants :

- Un `BorderPane` comme conteneur racine de la fenêtre principale.
- Une `MenuBar` positionnée en haut de la fenêtre, proposant les menus « Fichier » et « Aide ».
- Une barre de recherche, composée d'un `TextField` et de boutons « Rechercher » et « Réinitialiser ».
- Un `TabPane` central regroupant trois onglets : « Livres », « Emprunts » et « Statistiques ».
- Un `Accordion` contenant un `TitledPane` dédié au formulaire de saisie des livres.
- Un `GridPane` organisant de manière structurée les différents champs des formulaires.
- Des composants `TableView` pour l'affichage tabulaire des livres et des emprunts.
-

- Une barre d'état (status bar) en bas de la fenêtre, affichant des messages d'information contextuels.

L'utilisation du fichier FXML, combinée à l'outil Scene Builder, a permis une conception visuelle rapide de l'interface, tout en conservant une séparation nette entre la structure de la vue et la logique de traitement implémentée dans le contrôleur.

5.7 Fonctionnalité spécifique au projet – Export CSV

La fonctionnalité d'export CSV constitue l'un des points forts de notre application. Elle permet à l'utilisateur d'exporter, en un clic, l'ensemble du catalogue de livres vers un fichier texte au format CSV (Comma-Separated Values), exploitable par n'importe quel logiciel de tableur tel que Microsoft Excel ou LibreOffice Calc.

Techniquement, cette fonctionnalité repose sur le composant FileChooser de JavaFX, qui ouvre une boîte de dialogue native permettant à l'utilisateur de choisir l'emplacement de destination ainsi que le nom du fichier à générer. Une fois l'emplacement validé, la classe FileWriter est utilisée pour écrire séquentiellement les données dans le fichier : une première ligne d'en-tête précise le nom des colonnes exportées, suivie d'une ligne par livre contenant les valeurs correspondantes, séparées par des virgules.

À l'issue de l'export, une boîte de dialogue Alert confirme à l'utilisateur le succès de l'opération et précise le chemin du fichier généré, ou signale, le cas échéant, une erreur survenue lors de l'écriture (par exemple en cas de permission insuffisante sur le répertoire choisi).

Interfaces de l'application

Ce chapitre présente les différentes interfaces graphiques développées dans le cadre de ce projet. Pour chaque écran, nous décrivons les composants JavaFX mis en œuvre ainsi que les fonctionnalités qu'ils permettent à l'utilisateur d'accomplir.

L'ensemble des interfaces a été conçu dans un souci constant de cohérence visuelle et d'ergonomie. La feuille de style `style.css` définit une palette de couleurs harmonieuse, des espacements réguliers entre les composants ainsi que des effets visuels discrets (changement de couleur au survol des boutons, mise en évidence des champs en erreur) destinés à guider naturellement l'utilisateur dans l'accomplissement de ses tâches, sans recourir à une documentation externe.

6.1 Fenêtre principale

La fenêtre principale de l'application constitue le point d'entrée unique pour l'utilisateur. Elle s'organise autour d'un conteneur `BorderPane` structuré comme suit :

- MenuBar (zone haute) : regroupe les menus « Fichier » (Exporter CSV, Quitter) et « Aide » (À propos).
- Barre de recherche (sous la barre de menus) : un `TextField` accompagné de deux boutons, « Rechercher » et « Réinitialiser ».
- TabPane (zone centrale) : organise le contenu en trois onglets distincts — Livres, Emprunts et Statistiques.
- Barre d'état (zone basse) : affiche des messages contextuels (nombre de résultats trouvés, confirmation d'une opération, horloge en temps réel).

Cette organisation générale, conforme aux bonnes pratiques d'ergonomie logicielle, permet à l'utilisateur de toujours conserver un repère visuel stable tout en naviguant entre les différentes fonctionnalités de l'application.

ID	Titre	Auteur	ISBN	Catégorie	Année	Prix	Disponible
1	Le Petit Prince	Saint-Exupéry	978-2-07-040850-4	Roman	1943	12.5	true
2	1984	George Orwell	978-2-07-036822-8	Science-fiction	1949	15.9	false
3	L'Étranger	Albert Camus	978-2-07-036002-4	Roman	1942	11.5	true

6.2 Interfaces de gestion de l'entité 1 — Livres

L'onglet « Livres » regroupe l'ensemble des fonctionnalités liées à la gestion du catalogue. Il comprend un formulaire de saisie, intégré au sein d'un Accordion avec un TitledPane dédié, qui met en œuvre l'intégralité des seize contrôles JavaFX exigés par le cahier des charges du module

Contrôle JavaFX	Champ associé
TextField	Titre, Auteur, ISBN
TextArea	Description du livre
ComboBox	Catégorie du livre
Spinner	Année de publication, Nombre de pages
Slider	Prix, Note d'appréciation
CheckBox	Disponibilité du livre

Contrôle JavaFX	Champ associé
RadioButton + ToggleGroup	État (Neuf / Bon état / Usagé)
DatePicker	Date de publication
ColorPicker	Personnalisation visuelle (étiquette de catégorie)
Tooltip	Aide contextuelle sur chaque champ du formulaire

Tableau 6.1 – Contrôles JavaFX du formulaire « Livres »

Sous le formulaire figurent les boutons d'action « Ajouter », « Modifier », « Supprimer » et « Réinitialiser », ainsi qu'une zone de filtres combinant une ComboBox de catégories et une CheckBox de disponibilité. Enfin, un TableView affiche de manière synthétique l'ensemble des livres du catalogue, avec une colonne par attribut principal, permettant un tri et une consultation rapide.

ID	Titre	Auteur	ISBN	Catégorie	Année	Prix	Disponible
1	Le Petit Prince	Saint-Exupéry	978-2-07-040850-4	Roman	1943	12.5	true
2	1984	George Orwell	978-2-07-036822-8	Science-fiction	1949	15.9	false
3	L'Étranger	Albert Camus	978-2-07-036002-4	Roman	1942	11.5	true
4	le dernier jour d'un cond...	Victor Hugo	536548	Roman	1992	16.77419...	true

6.3 Interfaces de gestion de l'entité 2 — Emprunts

L'onglet « Emprunts » suit une logique similaire à celle de l'onglet « Livres », adaptée à la gestion des prêts. Le formulaire de saisie comprend :

- Une ComboBox listant uniquement les livres actuellement disponibles, afin d'éviter tout emprunt incohérent.
- Un TextField permettant de saisir le nom de l'emprunteur.
- Deux composants DatePicker, pour la date d'emprunt et la date de retour prévue.
- Un groupe de RadioButton, associé à un ToggleGroup, pour indiquer le statut de l'emprunt (En cours / Terminé).
- Une TextArea destinée à la saisie de remarques complémentaires (état du livre au retour, observations diverses).

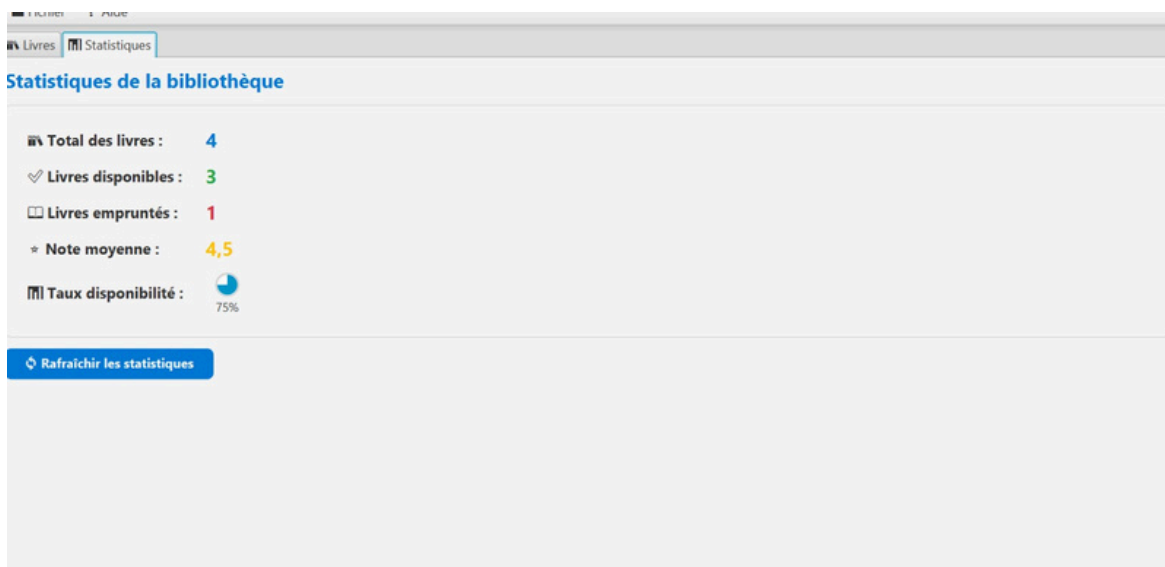
Comme pour les livres, des boutons « Ajouter », « Modifier », « Supprimer » et « Réinitialiser » permettent de gérer le cycle de vie des emprunts, tandis qu'un TableView affiche la liste complète des emprunts enregistrés, avec une mise en évidence visuelle des emprunts en retard.

6.4 Tableau de bord — Statistiques

L'onglet « Statistiques » offre une vision synthétique de l'état du fonds documentaire de la bibliothèque. Il présente notamment :

- Des indicateurs chiffrés affichés sous forme de Label : nombre total de livres, nombre de livres disponibles, nombre de livres actuellement empruntés et note moyenne du catalogue.
- Un ProgressIndicator représentant visuellement le taux de disponibilité des livres.
- Une ProgressBar illustrant la progression du chargement des statistiques depuis la base de données.
- Une ListView présentant la répartition du catalogue par catégorie, avec le nombre de livres associé à chacune.
- Un bouton « Rafraîchir » permettant de recalculer et de mettre à jour l'ensemble des indicateurs en temps réel.

Ce tableau de bord constitue un outil d'aide à la décision précieux pour le bibliothécaire, qui peut ainsi identifier rapidement les catégories sous-représentées ou suivre l'évolution du taux d'emprunt dans le temps.



6.5 Menus et dialogues

La MenuBar de l'application propose une organisation classique des fonctionnalités transverses :

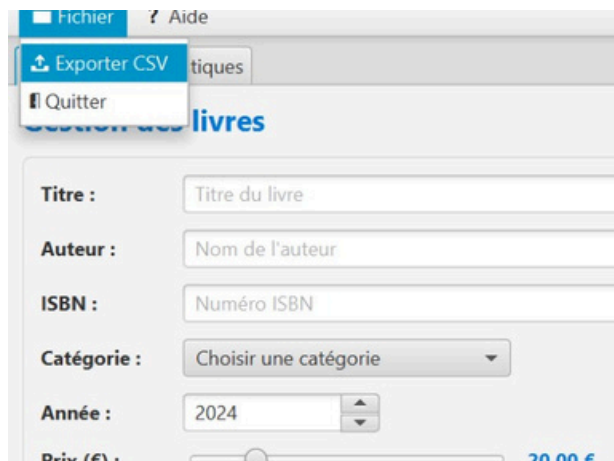
Fichier → Exporter CSV : déclenche l'ouverture d'un FileChooser pour l'export du catalogue.

Fichier → Quitter : ferme proprement l'application après libération des ressources (connexion à la base de données notamment).

Aide → À propos : affiche une fenêtre Alert présentant les informations relatives au projet (titre, auteurs, version, encadrante).

Les composants Alert sont systématiquement utilisés pour communiquer avec l'utilisateur à des moments clés de l'utilisation de l'application :

- Confirmation avant toute action destructrice ou importante (ajout, modification, suppression d'un livre ou d'un emprunt).
- Signalement d'erreurs de saisie, telles que des champs obligatoires laissés vides ou des formats de données invalides.
- Information de l'utilisateur sur le succès d'une opération réalisée (par exemple, confirmation de l'enregistrement d'un nouveau livre).

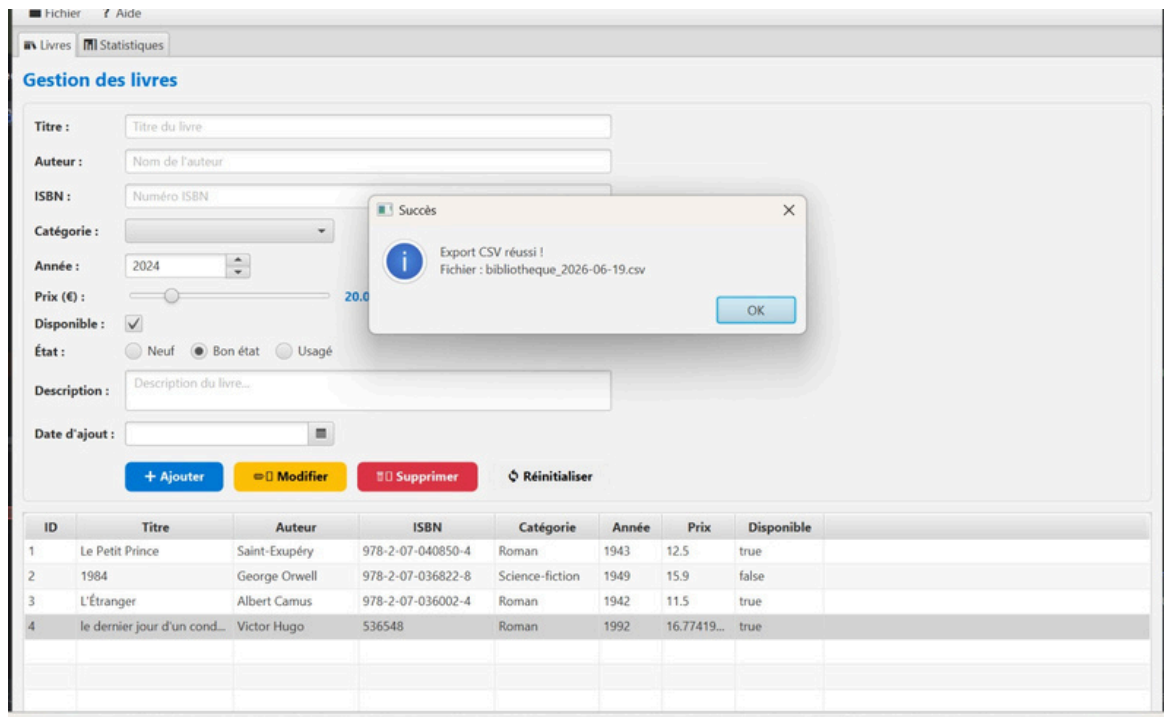


6.6 Export de données

La fonctionnalité d'export de données, accessible depuis le menu « Fichier », permet de générer un fichier CSV contenant les informations suivantes pour chaque livre du catalogue :

- Titre
- Auteur
- ISBN
- Catégorie
- Année de publication
- Prix
- État
- Disponibilité

Une fois l'export terminé, un message de confirmation est affiché à l'utilisateur, l'informant du chemin complet du fichier généré. Cette fonctionnalité illustre concrètement l'interopérabilité de notre application avec d'autres outils bureautiques couramment utilisés dans la gestion documentaire.



D'un point de vue ergonomique, l'ensemble des interfaces présentées dans ce chapitre a fait l'objet de plusieurs itérations successives, ajustées au fil des retours formulés par les deux membres du binôme lors des séances de test manuel. Les principaux ajustements ont porté sur la taille et l'espacement des composants, la cohérence des couleurs utilisées pour signaler les états (disponible/indisponible, en cours/terminé, en retard), ainsi que la clarté des messages affichés dans les boîtes de dialogue Alert, afin de garantir une expérience utilisateur fluide et sans ambiguïté.

6.7 Vidéo de démonstration

Afin d'illustrer le fonctionnement global de l'application, nous avons réalisé une vidéo de démonstration d'une durée de 2 minutes. Cette vidéo couvre les points suivants :

- Présentation rapide de l'interface (menus, onglets, navigation).
 - Démonstration de toutes les fonctionnalités CRUD (Livres et Emprunts).
 - Utilisation des contrôles JavaFX (Slider, Spinner, DatePicker, etc.).
- Recherche, filtrage et affichage des statistiques.
 - Export CSV.

Lien vers la vidéo de démonstration :

Tests utilisateur et démarche qualité

En complément des tests techniques décrits au chapitre précédent, une démarche de validation orientée utilisateur a été menée afin de s'assurer que l'application répondait effectivement aux attentes d'un usage réel en bibliothèque. Cette démarche s'est articulée autour de plusieurs scénarios représentatifs, exécutés par les deux membres du binôme en simulant le rôle d'un bibliothécaire.

7.1 Scénarios de test fonctionnels

Plusieurs scénarios ont été définis et exécutés de manière systématique afin de couvrir les principaux parcours utilisateur de l'application :

Scénario	Résultat attendu	Résultat obtenu
Ajout d'un livre avec données valides	Le livre apparaît dans le TableView et la base de données	Conforme
Ajout d'un livre avec champ titre vide	Une alerte d'erreur bloque l'enregistrement	Conforme
Recherche d'un livre par auteur partiel	Les résultats correspondants s'affichent dynamiquement	Conforme
Création d'un emprunt sur un livre indisponible	Le livre n'apparaît pas dans la ComboBox de sélection	Conforme
Modification du statut d'un emprunt en « Terminé »	La disponibilité du livre associé est remise à vrai	Conforme
Export CSV du catalogue	Un fichier CSV valide est généré à l'emplacement choisi	Conforme
Coupure du serveur MySQL pendant l'utilisation	Une alerte d'erreur informe l'utilisateur sans plantage	Conforme

Tableau 7.1 – Synthèse des scénarios de test fonctionnels

L'ensemble de ces scénarios s'est conclu de manière satisfaisante, confirmant la robustesse globale de l'application face aux cas d'usage courants ainsi qu'à certains cas limites volontairement provoqués.

7.2 Démarche de gestion de version

La nature des modifications apportées (par exemple : « Ajout de la méthode rechercher() dans LivreDAO » ou « Intégration du Slider pour le champ prix »). Quelques conflits mineurs sont néanmoins survenus, notamment lors de modifications simultanées du fichier MainController.java par les deux membres, et ont été résolus par une relecture conjointe du code avant validation de la fusion.

Le développement collaboratif du projet a été organisé autour d'un dépôt Git hébergé sur GitHub, selon une stratégie de branches simple mais efficace : chaque membre du binôme a travaillé sur une branche dédiée à son domaine de responsabilité (backend pour Kaoutar Baladi, frontend pour Fayza Belhaj), les fusions vers la branche principale étant réalisées après vérification du bon fonctionnement des modifications apportées.

Cette organisation a permis de limiter les conflits de fusion tout en conservant un historique clair de l'évolution du projet, chaque commit étant accompagné d'un message descriptif précisant la

7.3 Bilan de la démarche qualité

La combinaison des tests unitaires automatisés, des tests manuels orientés scénarios utilisateur et d'une gestion de version disciplinée a contribué à livrer une application stable et conforme au cahier des charges du module. Cette démarche, bien que modeste à l'échelle d'un mini-projet académique, reproduit les bonnes pratiques généralement attendues dans un contexte professionnel de développement logiciel.

Conclusion

Ce mini-projet réalisé dans le cadre du module Développement Java IHM nous a permis de mettre en pratique, de manière concrète et appliquée, l'ensemble des concepts théoriques abordés durant le semestre. Le développement de l'application « Gestion de Bibliothèque » nous a conduits à manipuler en profondeur la technologie JavaFX pour la conception d'interfaces graphiques, ainsi que la base de données MySQL pour la persistance des informations, le tout au sein d'une architecture logicielle structurée selon les patrons MVC et DAO.

Principaux acquis

- Maîtrise approfondie de JavaFX pour la conception d'interfaces graphiques riches et interactives.
- Compréhension et application concrète des patrons de conception MVC et DAO.
- Manipulation d'une base de données MySQL via JDBC, incluant la conception du schéma relationnel et l'écriture de requêtes SQL préparées.
- Collaboration efficace en équipe à l'aide de Git et GitHub, incluant la gestion des branches et des fusions.
- Capacité à résoudre des conflits techniques et à intégrer harmonieusement les contributions de chaque membre du binôme.

Difficultés rencontrées

- Gestion de certains conflits Git survenus lors de la fusion des branches de travail respectives des deux membres.
- Nécessité d'adapter et d'harmoniser les modèles de données entre le travail réalisé côté backend et son exploitation côté interface graphique.
- Configuration initiale de l'environnement de base de données avec WAMP, notamment au niveau des droits d'accès et de la création du schéma.
- Résolution de quelques problèmes de compatibilité liés à certaines versions de JavaFX lors de la mise en place de l'environnement de développement.

Perspectives d'amélioration

Bien que l'application réponde pleinement au cahier des charges fixé pour ce mini-projet, plusieurs pistes d'amélioration ont été identifiées pour de futures évolutions :

- Mise en place d'un système d'authentification afin de sécuriser l'accès à l'application selon les profils d'utilisateurs.
- Intégration de graphiques dynamiques (camemberts, histogrammes) pour enrichir le tableau de bord statistique.
- Ajout d'un module de gestion des adhérents, permettant un suivi individualisé des usagers de la bibliothèque.
- Développement d'une version web de l'application, accessible depuis un navigateur, afin d'élargir son accessibilité.
- Génération automatique de rapports au format PDF, synthétisant l'activité de la bibliothèque sur une période donnée.

En définitive, ce projet constitue une expérience formatrice riche, qui nous a permis de consolider nos compétences techniques en développement Java et de développer notre capacité à mener un projet logiciel de bout en bout, depuis l'analyse des besoins jusqu'à la livraison d'une application fonctionnelle, en passant par une collaboration structurée au sein d'un binôme.

Nous tenons enfin à souligner que ce mini-projet, bien que limité dans son périmètre fonctionnel par les contraintes calendaires du semestre, nous a donné un aperçu réaliste des étapes et des difficultés inhérentes à tout projet de développement logiciel : la nécessité d'une conception préalable rigoureuse, l'importance d'une communication constante au sein de l'équipe de développement, ainsi que la valeur d'une démarche de test systématique avant toute livraison. Ces enseignements, au-delà de la dimension purement technique, nous accompagneront tout au long de notre future carrière d'ingénieur en génie informatique.

Glossaire

Terme	Définition
CRUD	Acronyme désignant les quatre opérations de base sur les données : Create, Read, Update, Delete (créer, lire, mettre à jour, supprimer).
DAO	Data Access Object — patron de conception isolant l'accès aux données de stockage du reste de l'application.
MVC	Modèle-Vue-Contrôleur — patron d'architecture séparant les données, l'affichage et la logique de contrôle.
FXML	Format XML déclaratif utilisé par JavaFX pour décrire la structure d'une interface graphique.
JDBC	Java Database Connectivity — API Java standard permettant la connexion et l'interaction avec une base de données relationnelle.
ISBN	International Standard Book Number — identifiant numérique unique attribué à chaque ouvrage publié.
CSV	Comma-Separated Values — format de fichier texte tabulaire dont les valeurs sont séparées par des virgules.

Tableau – Glossaire des principaux termes techniques

Références

- 9.Documentation officielle JavaFX : <https://openjfx.io>
- 10.Documentation MySQL : <https://dev.mysql.com/doc/>
- 11.Documentation IntelliJ IDEA : <https://www.jetbrains.com/idea/>
- 12.Tutoriel JavaFX et MySQL : <https://www.codejava.net/javafx>
- 13.Pattern DAO : https://www.tutorialspoint.com/design_pattern
- 14.Git – Documentation officielle : <https://git-scm.com/doc>
- 15.WAMP Server : <https://www.wampserver.com/>
- 16.JavaFX Scene Builder : <https://gluonhq.com/products/scene-builder/>